Ders 11 Genişletilmiş Notu: Matrislerin Fonksiyonlara Aktarılması, Seçim Sistemleri, Sayarak Sıralama, Çoğunluk Elemanı ve Simetrik Matris

Ders 11 Genişletilmiş Notu: Matrislerin Fonksiyonlara Aktarılması, Seçim Sistemleri, Sayarak Sıralama, Çoğunluk Elemanı ve Simetrik Matris

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Fonksiyon Yazmanın Ölçüsü
- 2. Pointer ve Dizilerin Kısa Tekrarı, İlk Değer Verme
- 3. Matrislerin Fonksiyonlara Gönderilmesi
- 4. Seçim Sistemi Örnekleri
- 5. Sayarak Doğrusal Sıralama
- 6. Doğrudan Yerleştirerek Sıralama
- 7. Dizinin Ters Çevrilmesi
- 8. Çoğunluk Elemanı (Majority Element)
- 9. Transpoz, Döndürme ve Aynalama
- 10. İkili Matrisi Simetrik Yapmak

1. Fonksiyon Yazmanın Ölçüsü

Ders bir öğrencinin sorusuyla başladı.

Soru: Fonksiyon yazmanın ölçüsü nedir? Her kod tekrarı olan yerde fonksiyon yazmak zorunlu mu? Fonksiyon yazmak programı optimize eder mi?

Cevap: Hocaya göre fonksiyonlar programlama ve modelleme açısından en uygun tasarım aracıdır, çünkü programı modüllere ayırır. Fakat her şey için fonksiyon yazılmaz. `printf` zaten kendisi bir fonksiyondur. Bir iki satırlık ufak işleri ayrıca fonksiyon yapmaya gerek yoktur.

Fonksiyon yazılması gereken yer, programda sürekli çağrılabilen alt program parçalarıdır (subroutine). Bir dizinin sıralanması ya da elemanlarının yazdırılması böyledir. Fonksiyon yazılmazsa bu kodlar büyük ihtimalle programın birkaç yerinde tekrar eder. Tekrar eden kod (duplicated code) iki açıdan kötüdür:

- **Kod kalabalığı:** Aynı satırlar programın farklı yerlerinde birikir.
- **Değişiklik zorluğu:** Hocanın örneğiyle, dizi yazdırılırken araya yıldız koymak istediniz. Kodu kopyala yapıştır ile çoğalttıysanız her yerini tek tek değiştirmeniz gerekir. Fonksiyon yazdıysanız yalnız fonksiyonun içini değiştirmek yeter. Bu da kod geliştirmeyi hızlandırır.

Ölçü olarak hoca şunu söyledi: Beş altı satırlık bir kod bile iki ya da daha fazla kez tekrar ediyorsa fonksiyonla yazılmalıdır.

Optimizasyon tarafında ise bir bedel vardır. Fonksiyon çağrısı, programın akışının kesilip başka bir alt programın çağrılması ve bu kesinti sırasında bazı yedekleme işlemlerinin ya-

pılması demektir. Sistem programlama ya da alt seviye (low level) programlama yapıyorsak ve söz konusu kod bir kez çalışıp tekrar etmeyecekse fonksiyon yazmaya çok gerek yoktur.

Bunun karşısında nesneye yönelik programlama vardır. Java'da ya da C#'ta `getIdNumber` gibi bir sayıyı elde etmek için bile ayrı fonksiyon yazılır. Modülerliğe bu kadar önem verilir, çünkü yazılım mühendisliği tarafında yapılan iyi bir planlama, kodun hızlı çalışmasının önüne geçer. Günümüz derleyicileri ve işlemcileri hızlandığı için fonksiyon çağrıları eskisine göre çok daha az hız sorunu oluşturur. Alt seviyede ise duruma göre düşünmek gerekir.

C'de makrolar da vardır. `#define MAX 3` gibi yazdıklarımız aslında makrodur ve kodu kısaltır. Hoca bunların yapısal programlama konusu olduğunu söyleyerek ayrıntısına girmedi.

Ders başlamadan önce hoca bir dosya kopyalanmasını bekliyordu ve bunu şakayla bağladı: Derin kopyalama (deep copy) zor iştir, bir şeyi bir yerden bir yere kopyalamak bayağı zaman alır. Bu yüzden sığ kopyalama (shallow copy) pointer kullanımına daha uygundur.

2. Pointer ve Dizilerin Kısa Tekrarı, İlk Değer Verme

Son derste pointerların çoğu anlatılmıştı. Hoca kısaca hatırlattı:

- Bir `x` değişkeninin pointer değişkeni, değişkenle aynı türde ve önünde yıldız işaretiyle tanımlanır: `int *px`.
- Değişkenin adresi `&` operatörüyle alınır. Hoca buna referans alma (referencing) dedi: `px = &x`.
- `*px` ise dereferans işlemidir: `px`'in gösterdiği yerdeki değeri alır ya da değiştirir.
- Pointer dizilerle de kullanılır. Dizinin üzerinde dolaşmak gerektiği için pointer aritmetiği devreye girer.

Hoca bu arada dizilere ilk değer vermenin bir özelliğini ekledi. Şu tanımda ilk üç eleman 1, 2, 4 olur, kalanlar otomatik olarak 0 olur:

```
int dizi[10] = {1, 2, 4};
```

Bütün diziyi sıfırlamak için başka bir şey kullanmaya gerek yoktur. İlk elemanı 0 yapmak, diğer elemanları da 0 yapmak anlamına gelir: `int dizi[10] = {0};`. Bazı yerlerde bunun için `static` kullanılır, ama `static` başka amaçlarla da kullanıldığı için hoca karışmasın diye diziyi doğrudan bu şekilde sıfırlamayı önerdi.

Matrislerde de aynısı geçerlidir. 5'e 5'lik bir matrisi her satır için ayrı ayrı `{0, 0, 0, 0, 0}` yazarak ilklendirebilirsiniz, ama doğrudan şöyle de yazabilirsiniz:

```
int M[5][5] = {0};
```

3. Matrislerin Fonksiyonlara Gönderilmesi

3.1 Önerilen Yol

Bir `yazdir` fonksiyonumuz olsun. Ona matrisi ve boyutlarını göndereceğiz. Hoca fonksiyonun tanımında ilk parametrenin nasıl yazılacağını sordu. Öğrencilerden gelen cevapların içinden doğrusunu şöyle gösterdi: Satır sayısı yazılmadan, sütun bilgisi yazılır. Sütun sayısının yerine `#define` ile tanımlanmış bir değer de kullanılabilir. Çağrıya bakınca iki tamsayı daha geleceği görülür, onlar da `m` ve `n` parametreleri olur:

```
#include <stdio.h>

void yazdir(int M[][5], int m, int n)
{
    int i, j;
    for (i = 0; i < m; i++) {
        for (j = 0; j < n; j++)
            printf("%3d", M[i][j]);
        printf("\n");
    }
}
```

```c
int main(void)
{
    int M[5][5] = {0};
    yazdir(M, 5, 5);
    return 0;
}
```

Hoca elemanların yan yana değil matris biçiminde görünmesi için iç döngüden sonra `\n` ekledi. Çıktı beş satır beş sütun sıfırdır.

Hoca bu sırada, adlandırma karışmasın diye matrise büyük `M`, satır sayısına küçük `m` dediğini belirtti. Ayrıca kodların C dosyası olarak kaydedilmesini istedi. C++ C'yi kapsıyor gibi görünse de derleyiciler bakımından farklılık vardır.

3.2 Pointer Aritmetiğiyle Denemeler

Aynı işi pointer aritmetiğiyle yazmak için hoca birkaç deneme yaptı. Parametre, dizinin ilk elemanının adresini tutan bir pointer olarak yazıldı ve köşeli parantezlerle `M[i + j]` kadar ilerlemek denendi. Hocanın anlatımına göre `i` ve `j` sıfırken sıfırıncı satırdayız, `j` 1 olunca sıfırıncı satırın birinci hücresine gideriz.

> **Ders sonrası not:** `M[i + j]` satır atlamaz. Örneğin `i = 1, j = 0` ile `i = 0, j = 1` aynı hücreyi verir. Matris tek bir `int *` ile gönderilirse, büyük olasılıkla doğru yazım `&M[0][0]` ile çağırıp elemana `M[i * n + j]` ile erişmektir. Hoca da az sonra satıra giderken "orada bir çarpma işlemi var" dedi.

Derleme sırasında önce "conflicting types for yazdir" hatası çıktı. Hoca, pointer kullanınca prototip yazmak gerektiğini söyleyerek fonksiyonun prototipini ekledi.

> **Ek bilgi (derste anlatılmadı):** Bu hata büyük olasılıkla pointer kullanmaktan değil, fonksiyonun tanımından önce çağrılmasından doğar. Derleyici tanımını görmediği bir fonksiyonu eski C kurallarıyla `int` döndürür kabul eder. Sonra gelen `void` tanım bununla çakışır. Prototip bu çakışmayı önler.

Ardından uyumsuz dönüşüm uyarısı geldi. Hoca, `M`'ye `i` ekleyip o yerdeki değeri almayı denedi. Program çalışınca her satırda 0 1 2 3 4 yazdı, oysa matris sıfırlardan oluşuyordu.

Soru: Yıldız parantezin bir dışında mı olacak?

Cevap: Hoca, ifadenin o haliyle gidip oradaki değeri getirdiğini ve o değere `j` eklediğini gördü. Parantezler yüzünden farklı bir şey alınıyordu. Yıldız dışarı alınınca doğru sonuç çıktı. Son hali şudur:

```c
printf("%3d", *(*(M + i) + j));
```

Hoca bunu şöyle okudu: `M` adresine git, oradan `i` kadar satır ilerle (bunun içinde sütun sayısıyla bir çarpma vardır). Elde edilen satırda `j`'inci sütuna git ve o adresteki değeri getir. Bu yazım `int M[][5]` parametresiyle `gcc -Wall` altında uyarısız derlendi.

Pointer ile matrisin her elemanının adresini elde edip o adresleri fonksiyonda ayrı ayrı kullanmak gibi birçok algoritma geliştirilebilir. Hoca bunların derinine yapısal programlama dersinde girileceğini söyledi. Bu dersin kapsamı için önerisi nettir:

> **Hocanın vurgusu:** Yapısal programlamaya giriş kapsamında bir matrisi fonksiyona göndermek için birinci yazımı, yani `int M[][5]` biçimini kullanın. Pointer kullanmaya hiç gerek yoktur.

Diziler için de geçen dersteki durum hatırlatıldı: Fonksiyona dizinin adı ya da sıfırıncı elemanının adresi gönderilebilir. Bunların hepsi adres olduğu için fonksiyon tarafında pointer olarak alınır. Hoca konunun basit olduğunu, kuralların üç beş tane olduğunu, ama bunlar birleştirilince çok karışık işlemlerin yapılabildiğini, bu yüzden bol uygulama gerektiğini söyledi.

4. Seçim Sistemi Örnekleri

Son dersten kalan seçim sistemi örneklerine devam edildi. Hoca bunları kod yerine akış adımlarıyla anlattı, çünkü asıl mesele veri yapısının seçimidir.

Aşağıda hocanın tahtadaki gibi indisler 1'den başlar. C'de dizi 0'dan başladığı için kod yazarken buna göre ayarlama yapılır. Vatandaş sayısı V, parti sayısı P ile gösterilir. Hoca oy dizisine de zaman zaman "P dizisi" dedi. Karışmasın diye burada bu diziye `parti` deniyor.

4.1 Birinci Durum: Her Vatandaş Bir Partiye Oy Verir

Bu, her öğrencinin bir sorunun cevabını işaretlemesine de benzer. Değişkenlerle çözmek zordur, çünkü V tane vatandaş için V tane değişken gerekirdi. Daha kolay saklama sağlayan yapı dizidir. Önce dizinin indislerine ve değerlerine bir anlam verilir:

- `vatandas` dizisinde indis vatandaşın numarasıdır. `vatandas[i]`, i. vatandaşın oy verdiği partidir, yani 1 ile P arasında bir değerdir.
- `parti` dizisinde indis partinin numarasıdır. `parti[j]`, j. partinin oy sayısıdır. Başlangıçta bütün değerler 0'dır.

Algoritmanın adımları şöyledir:

1. `parti` dizisini sıfırla.
2. $i = 1$'den V'ye kadar (V dahil) birer birer git.
3. Her i için `parti[vatandas[i]]++` yap.
4. Dur.

Burada elde edilen değer doğrudan indis olarak kullanılır. Hoca, daha önce bunun iç içe döngüyle de yapıldığını ve o yolun kötü olduğunu hatırlattı. Kazanan, `parti` dizisindeki en büyük değere sahip partidir.

4.2 Yatay Histogram

Hoca, partilerin oylarının 8, 6, 3, 5, 7, 0 çıktığını varsaydı ve bunun histogramının nasıl çizileceğini sordu. Düşey histogram biraz daha karmaşık olduğu için yatay çizildi. Öğrencilerin dediği gibi iki iç içe döngü yeterlidir:

1. $i = 1$'den P'ye kadar git.
2. Her i için $j = 1$'den `parti[i]`'ye kadar git ve bir yıldız bas.
3. İç döngü bitince alt satıra geç.

Bu örneğin çıktısı şöyledir (ara adım eklendi):

```
********
******
***
*****
*******
```

Son satır boştur, çünkü altıncı partinin oyu 0'dır.

Soru: Neden histogram çizdiriyoruz?

Cevap: Histogram bir görselleştirme aracıdır (data visualization). Diziye bakıp hangisinin daha fazla oy aldığını görmek zor olabilir. Ama yüz tane değer varsa, grafikte en yükseği çok daha hızlı görürsünüz.

4.3 İkinci Durum: Her Vatandaş K Partiye Oy Verir

Bir vatandaş iki, üç, dört partiye oy verebiliyorsa dizi işe yaramaz, matris gerekir.

> **Hocanın vurgusu:** Hangi veri yapısını kullanacağımızı tespit etmek, en başta yapmamız gereken iştir.

Matrisin satır ve sütunlarına anlam verilir. Satırlar vatandaşlar ($1 \ldots V$), sütunlar oylar ($1 \ldots K$) olsun. `O[i][j]`, i. vatandaşın j. oyunu verdiği partidir, yani yine 1 ile P arasında bir sayıdır. Örneğin birinci vatandaş birinci oyunu 3 numaralı, ikinci oyunu 4 numaralı partiye vermiş olabilir. Burada puan yoktur, oylar yalnızca sayılır. K'nin de sınırlı verilmesi gerekir.

Bu matrisin transpozu da aynı işi görür (satırlar oylar, sütunlar vatandaşlar). Hocaya göre ikisi arasında hiçbir fark yoktur, bu tamamen bir tasarım tercihidir.

Adımlar şöyledir:

1. `parti` dizisini sıfırla.
2. $i = 1$'den V'ye kadar git (vatandaşlar).
3. Her i için $j = 1$'den K'ye kadar git (oylar).
4. `parti[O[i][j]]++` yap.
5. Dur.

Kazanan yine `parti` dizisinin en büyük değeridir.

4.4 Üçüncü Durum: Her Vatandaş Her Partiye Puan Verir

Şimdi her vatandaş her partiye belli bir aralıkta, örneğin 1 ile 100 ya da 1 ile 10 arasında puan verir. Yine matris gerekir, ama önceki matristen farklıdır. Satırlar vatandaşlar ($1 \ldots V$), sütunlar partilerdir ($1 \ldots P$). `O[i][j]`, i. vatandaşın j. partiye verdiği puandır. Sohbette "vatandaş sayısı çarpı 2" boyutu önerilmişti. Hoca bunun olmayacağını, ikinci boyutun P olduğunu söyledi. Transpozu da yine kullanılabilir.

Veri girişi sırasında puanların aralıkta olması `if` ya da `do-while` ile denetlenebilir. Aralık 100'e kadarsa kullanıcı 101 girememelidir. Hoca bu sistemin herkesin herkese belli bir puan verebilmesi bakımından daha adil göründüğünü, hatta eksi oyun bile düşünülebileceğini, ama bunun siyaset bilimcilerin alanı olduğunu söyledi.

Toplam için bir `puan` dizisi tutulur. `puan[j]`, j. partinin toplam puanıdır ve başlangıçta 0'dır:

1. $i = 1$'den V'ye kadar git.
2. Her i için $j = 1$'den P'ye kadar git.
3. `puan[j] += O[i][j]` yap.

Döngüler bitince birinci vatandaşın verdiği bütün puanlar, sonra ikinci vatandaşınkiler partilerin hanesine eklenmiş olur. Bu problem için bir matris ve bir dizi kullanılmıştır.

> **Hocanın vurgusu:** Matrisi herkes tasarlayabiliyor. Ama bazı arkadaşlar sonuçları toplarken önce bir matris daha kuruyor, topluyor, çıkarıyor, sonra diziye çeviriyor. Problem diziyle çözülebiliyorsa ve siz matrisle çözüyorsanız iyi bir algoritma tasarlamadınız demektir.

4.5 Kazananın Bulunması ve Eşitlik

Bütün durumlarda kazanan aynı şekilde bulunur: dizinin en büyük değeri. Aynı puanı alan partiler de olabilir. O zaman iki geçiş yapılır:

1. Dizinin en büyük değeri bulunur. Bu n adım sürer.
2. Dizi bir kez daha gezilir (n adım daha). En büyük değere eşit birden fazla parti varsa hepsi kazanmıştır, yani koalisyon olur.

İki parti olması da gerekmez. 100 partinin 10 tanesi aynı puanı aldıysa 10 partili bir koalisyon demektir.

4.6 Matrissiz Çözüm Önerisi

Soru: İkinci ve üçüncü örnekler matris kullanmadan yapılamaz mı? Bir dış döngüyle vatandaşları gezip iç döngüde her partinin puanını kullanıcıdan alsak, girilen değeri doğrudan parti dizisine eklesek olmaz mı?

Cevap: Hoca, programlama açısından olabileceğini söyledi. Üçüncü vatandaşın verdiği puanlar gelir ve parti dizisine dağıtılır. Fakat iş bitince üçüncü vatandaşın hangi partiye ne verdiği bilgisi kaybolur. Sadece partilerin toplam oyları isteniyorsa bu çözüm kullanılabilir. Bilgiler saklansın isteniyorsa matris daha doğrudur. Dolayısıyla öneri yanlış değildir, problemin ne istediğine göre düşünmek gerekir.

4.7 Dördüncü Durum: Puanlar Bir Ölçüte Göre Ağırlıklandırılır

Son durumda vatandaşın verdiği puan matrise doğrudan yazılmaz. Kullanıcıdan alınan puan p önce bir f fonksiyonundan geçer, dönen sonuç matrise aktarılır:

$$O[i][j] = f(p)$$

Toplama ve en büyüğü bulma adımları üçüncü durumla aynıdır. Fonksiyonun içine ne yazılacağını hoca öğrencilere sordu. Sohbetten IQ seviyesi, kazanılan para, eğitim derecesi gibi öneriler geldi. Örneğin puan, kişinin IQ'su ile doğru orantılı olacak şekilde $\frac{IQ}{100}$ gibi bir katsayıyla çarpılabilir. Hoca bu tarafı çok önemsemediğini, burada yalnızca sayısal düşündüklerini belirtti. Önemli olan, fonksiyonun puanı farklı bir değere dönüştürmesidir. Bu durumda değerler kesirli olabileceği için matris ve puan dizisi `float` türünde olur.

5. Sayarak Doğrusal Sıralama

Aradan sonra hoca iki sıralama algoritmasına geçti. Şimdiye kadar selection sort ve bubble sort görülmüştü. Birincisinin adı "linear sort with counting" idi. Fikri şudur:

> Bir elemandan küçük k tane eleman varsa, o elemanın sıralı dizideki yeri $k + 1$'dir.

5.1 Elle Örnek

Dizi `A = 3, 1, 4, 8` olsun. Her eleman için kendisinden küçük kaç eleman olduğu bulunur ve bir K dizisinde tutulur. Yeri ise $K + 1$'dir (burada indisler C'deki gibi değil, 1, 2, 3, 4 diye düşünülür):

Eleman	Kendisinden küçükler	K	Yeri ($K + 1$)
3	1	1	2
1	yok	0	1
4	3 ve 1	2	3
8	3, 1 ve 4	3	4

Yeni bir B dizisinde 3 ikinci yere, 1 birinci yere, 4 üçüncü yere, 8 dördüncü yere yazılır. Sonuç `1, 3, 4, 8` olur.

5.2 Birinci Sürümün Kodu

Hoca kodu `main` içinde yazdı. `10` sayısını her yere yazmamak için `#define` ile `MAX` tanımladı, sonra hızlı deneme yapmak için onu 5'e indirdi. Üç dizi vardır: girilen `dizi`, her elemandan küçüklerin sayısını tutan ve sıfırlanan `kucuk`, sıralı sonucun yazılacağı `yeni`.

İç içe iki döngü gerekir. Hangi elemanın nerede olduğunu bilmediğimiz için `i` de `j` de 0'dan başlar. `dizi[i]`, `dizi[j]`'den büyükse `kucuk[i]` bir artar. C'de indis 0'dan başladığı için $k + 1$ yapmaya gerek kalmaz, yer doğrudan `kucuk[i]`'dir:

```c
#include <stdio.h>
#define MAX 5

int main(void)
{
    int dizi[MAX], kucuk[MAX] = {0}, yeni[MAX];
    int i, j;

    for (i = 0; i < MAX; i++) {
        printf("Eleman giriniz: ");
        scanf("%d", &dizi[i]);
    }
    for (i = 0; i < MAX; i++) {
        for (j = 0; j < MAX; j++) {
            if (dizi[i] > dizi[j]) {
                kucuk[i]++;
            }
```

```
        }
    }
    for (i = 0; i < MAX; i++) {
        yeni[kucuk[i]] = dizi[i];
    }
    for (i = 0; i < MAX; i++) {
        printf("%3d", yeni[i]);
    }
    printf("\n");
    return 0;
}
```

Derste girilen `8 4 5 3 99` dizisi `3 4 5 8 99` olarak sıralandı.

5.3 Bu İyi Bir Algoritma mı?

Hoca bunu sorunca öğrencilerden üç eleştiri geldi.

Doğruluk (correctness): Aynı elemanlar girilirse algoritma düzgün çalışmaz. Eşit iki elemanın `kucuk` değeri aynı çıkar, ikisi de `yeni` dizisinde aynı yere yazılır ve bir hücre boş kalır. `3 5 3 2 6` dizisinde her iki 3'ün `kucuk` değeri 1'dir (ara adım eklendi). Program derlenip çalıştırıldığında `yeni[2]` hiç yazılmadı.

Hoca çözüm olarak karşılaştırmaya eşitliği eklemeyi denedi (`>=`). Derste girilen `3 5 3 2 6` için program dizinin dışına taştı. Hoca sebebini de gördü: Sıfırıncı eleman artık hiç bulunamıyor, her eleman kendisini de saydığı için en küçük yer `1` oluyor. Değerler şöyle çıkar (ara adım eklendi):

Eleman	3	5	3	2	6
`>` ile `kucuk`	1	3	1	0	4
`>=` ile `kucuk`	3	4	3	1	5

Son satırdaki 5 değeri, beş elemanlı dizide olmayan `yeni[5]` hücresidir. Hoca bunun üzerine biraz kafa yormak gerektiğini söyledi.

İşlem sayısı: Sohbette bir öğrenci $n^2 + n$ işlem olduğunu yazdı. Hoca onayladı. Selection sort ve bubble sort da zaten n^2 işlem yapıyordu. Bu algoritma n^2'yi n'e düşürseydi anlaşılırdı, ama yine n^2'dir.

Bellek: Normal dizinin yanında `kucuk` ve `yeni` diye iki n elemanlı dizi daha gerekir.

Sonuç olarak hem işlem hem bellek açısından bubble sort ya da selection sort daha iyidir.

5.4 İkinci Sürüm: Her Çifti Bir Kez Karşılaştırmak

İşlem sayısını azaltmak için hoca, n^2'yi yaklaşık $\frac{n^2}{2}$'ye indirmeyi önerdi. Her seferinde sıfırıncı elemandan bakmak yerine `j, i + 1`'den başlar. Çünkü önceki elemanlarla karşılaştırma zaten yapılmıştır. Bir karşılaştırmanın iki sonucu da kullanılır: `dizi[i]` büyükse `kucuk[i]`, değilse `kucuk[j]` artar. Hoca bunu "bir oradan, bir buradan hallediyoruz" diye anlattı, `if` ve `else` bloklarından biri kesin çalışır. Değişen bölüm şudur:

```
for (i = 0; i < MAX; i++) {
    for (j = i + 1; j < MAX; j++) {
        if (dizi[i] > dizi[j]) {
            kucuk[i]++;
        }
        else {
            kucuk[j]++;
        }
    }
}
```

5.5 İki Sürümün Analizi

Hoca iki sürümde kaç işlem yapıldığını hesapladı. Döngünün içine girince kesin yapılan işlem `if` karşılaştırmasıdır. Buna temel işlem (basic operation) denir ve maliyeti 1 birim kabul edilir. Döngü demek toplam sembolü demektir. `MAX` yerine n yazılır. `<` kullanıldığı için üst sınır dahil değildir, toplam $n-1$'e kadar gider.

Birinci sürümde terimler `i`'ye ve `j`'ye bağlı değildir. Terim sayısı kadar sabitin çarpımı alınır:

$$\sum_{i=0}^{n-1}\sum_{j=0}^{n-1} 1 = \sum_{i=0}^{n-1} n = n \cdot n = n^2$$

İkinci sürümde iç toplamın terim sayısı $(n-1)-(i+1)+1 = n-1-i$ olur. Bu `i`'ye bağlı olduğu için toplam iki parçaya ayrılır:

$$\sum_{i=0}^{n-1}\sum_{j=i+1}^{n-1} 1 = \sum_{i=0}^{n-1}(n-1-i) = \sum_{i=0}^{n-1}(n-1) - \sum_{i=0}^{n-1} i$$

Birinci parça n tane $(n-1)$'in toplamıdır, yani $n(n-1)$'dir. İkinci parça 0'dan $n-1$'e kadar olan sayıların toplamıdır, yani $\frac{n(n-1)}{2}$'dir. Buradan:

$$n(n-1) - \frac{n(n-1)}{2} = \frac{n(n-1)}{2} \approx \frac{n^2}{2}$$

Hoca bunu tahtada bir kareyle gösterdi. n^2'lik algoritma kare matrisin bütün hücrelerini tarar. $\frac{n^2}{2}$'lik algoritma ise yalnızca bir üçgeni tarar, yani yarı yarıya bir tarama yapar. Ek bellek ihtiyacı değişmez.

Hoca bellek ve aynı elemanlar açısından kötü durumun sürdüğünü, bu yüzden bunun çok kullanılan bir sıralama olmadığını söyledi.

> **Ders sonrası not:** Hoca "aynı eleman olursa kötü durum var" derken büyük olasılıkla birinci sürümdeki sorunu kastediyordu. İkinci sürümde bu sorun yoktur. Eşit elemanlarda karşılaştırma `else` koluna düşer ve sonraki elemanın sayacı artar, böylece eşit elemanlar farklı yerlere yazılır. İkinci sürüm derlenip çalıştırıldı: `3 5 3 2 6` girdisi `2 3 3 5 6` verdi, `8 4 5 3 99` girdisi `3 4 5 8 99` verdi. İki kod da `gcc -Wall -std=c89 -pedantic` ile uyarısız derlendi.
>
> **Ek bilgi (derste anlatılmadı):** Dersteki ad "linear sort" olsa da bu algoritma n^2 karşılaştırma yapar. Bu yöntem kaynaklarda büyük olasılıkla karşılaştırma sayarak sıralama (comparison counting sort) adıyla geçer. "Counting sort" diye bilinen algoritma ise büyük olasılıkla karşılaştırma yapmaz, değer aralığı üzerinden sayar ve bir sonraki bölümdeki yönteme daha yakındır.

6. Doğrudan Yerleştirerek Sıralama

İkinci algoritmayı da hoca "linear sort" olarak andı. Bir A dizisi verilsin. Algoritma şunu yapar:

1. Dizinin en büyük elemanını bul.
2. 0'dan bu en büyük değere kadar indisleri olan yeni bir B dizisi aç.
3. B'nin bütün elemanlarını -1 yap. Elemanların 0 ya da pozitif olduğu kabul edilir, 0 da kabul edileceği için başlangıç değeri 0 değil -1 seçilir.
4. Her elemanı kendi değerindeki indise kopyala: `B[A[i]] = A[i]`.
5. B'yi baştan sona gez. -1 olmayan değerleri bir k işaretçisiyle sırayla A'nın üzerine yaz.

Hocanın örneğinde elemanlar 3, 8, 6, 7, 5'tir. En büyük değer 8 olduğu için B 0'dan 8'e kadar 9 elemanlıdır. Yerleştirme bittikten sonra (ara adım eklendi):

İndis	0	1	2	3	4	5	6	7	8
`B`	-1	-1	-1	3	-1	5	6	7	8

-1 olmayanlar sırayla alınınca A dizisinin yeni hali `3, 5, 6, 7, 8` olur. En büyük eleman bulunmadan bu algoritma yapılamaz.

6.1 Hız ve Bedeli

Hoca algoritmanın aslında çok hızlı olduğunu söyledi: En büyüğü bulmak n adım, `B[A[i]] = A[i]` atamaları n adım, -1 dışındakileri geri yazmak n adım. Yani en fazla $3n$ gibi bir sayı, n mertebesinde bir işlem. n^2'ye çıkılmaz.

Ama ciddi sorunları vardır:

- **Bellek:** Elemanlardan biri örneğin 3, 8, 999 değil de $10^6 - 5$ ise, B dizisi gereksiz yere çok uzun olur. Korkunç miktarda bellek ayırmak (allocation) gerekebilir. Sıkıntı, içerideki en büyük değerin büyüklüğünden kaynaklanır.
- **Aynı değerden birden fazla eleman:** Hoca, en büyük değerden birden fazla olabileceğini de sorun olarak saydı. Aynı değerdeki elemanlar `B`'de aynı hücreye yazılır.

Dizinin çalışma anında açılmasını (dinamik bellek) gelecek dönem görecekler. Şimdilik elemanların 0 ile `int`'in en büyük değeri arasında olduğu ve dizi boyutunun da buna göre tanımlandığı kabul edilebilir.

> **Ders sonrası not:** Hoca $10^6 - 5$ değerini derste net söyleyemedi. Bu değer 999995'tir, yani B dizisinin 999996 elemanı olur. Ayrıca hocanın $3n$ sayımında büyük olasılıkla dizinin boyutu ile en büyük değer ayrımı atlanmıştır. B'nin -1 yapılması ve baştan sona gezilmesi n adım değil, en büyük değer M için $M + 1$'er adım sürer. Toplam yaklaşık $2n + 2(M + 1)$ olur. M, n'den çok büyük değilse bu n mertebesindedir. Bellekle birlikte süre de M'ye bağlıdır.

Hoca algoritmanın kodlanmasını ve bu sorunlar için çözüm düşünülmesini istedi. İpucu olarak bunu önceki bölümdeki sayarak sıralamayla birleştirip melez (hibrit) bir çözüm yapılabileceğini söyledi.

7. Dizinin Ters Çevrilmesi

Bundan sonra uygulama saati için asistanların hazırladığı sorulara geçildi. İlk soru bir dizinin tersini bulmaktı.

7.1 Üç Çözüm

En basit yaklaşım yeni bir dizi oluşturup elemanları sondan başa doğru kopyalamaktır. Ek bellek ve n işlem gerektirir.

Derste daha önce yazılan yol `for` döngüsüyle yer değiştirmedir (swap). i. elemanla sondan i. eleman yer değiştirir:

```
for (i = 0; i < n / 2; i++) {
    temp = dizi[i];
    dizi[i] = dizi[n - i - 1];
    dizi[n - i - 1] = temp;
}
```

Burada -1 sorusu soruldu. C'de diziler 0'dan $n - 1$'e kadardır. -1 yazılmazsa `dizi[n - i]` olur ve $i = 0$ için dizinin dışına yazılmaya çalışılır.

Asistanların farklı ve daha kompakt yaklaşımı `while` döngüsü kullanır. Hoca bunun ikili arama mantığına benzediğini söyledi. Biri baştan, biri sondan gelen iki indis vardır:

```
left = 0;
right = n - 1;
while (left < right) {
```

```
    t = dizi[left];
    dizi[left] = dizi[right];
    dizi[right] = t;
    left++;
    right--;
}
```

Hoca iki indis eşit olunca orta noktaya gelindiğini ve durulduğunu söyledi. İki parça da `gcc -Wall` ile derlenip denendi ve dizileri doğru ters çevirdi.

7.2 Kaba İşlem Sayımı

Hoca iki sürümün işlem sayısını yaklaşık olarak saydı. Her dizi erişimini, atamayı ve aritmetik işlemi 1 saydı.

`for` sürümünde döngü gövdesi $\frac{n}{2}$ kez döner. Bir turdaki işlemler şöyledir:

- `temp = dizi[i]`: bir dizi erişimi ve bir atama, toplam 2.
- `dizi[i] = dizi[n - i - 1]`: iki aritmetik işlem ve bir dizi erişimi (3), bir atama (1), bir dizi erişimi (1), toplam 5.
- `dizi[n - i - 1] = temp`: iki aritmetik işlem ve bir dizi erişimi (3), bir atama (1), toplam 4.

Toplam 11 işlem eder, yani $11 \cdot \frac{n}{2}$. Döngü denetimi de vardır: `i++`, `n / 2` hesabı ve `<` karşılaştırması. Bunlar $\frac{n}{2} + 1$ kez yapılır. Başlangıçtaki `i = 0` de 1 işlemdir:

$$11 \cdot \frac{n}{2} + 3\left(\frac{n}{2} + 1\right) + 1$$

`while` sürümünde bir turda `t = dizi[left]` 2, `dizi[left] = dizi[right]` 3, `dizi[right] = t` 2, `left++` ve `right--` 2 işlemdir. Toplam 9 işlem eder. `left` ve `right` tam ortaya kadar geldiği için burada da gizli bir $\frac{n}{2}$ vardır. Başlangıç işlemleriyle birlikte:

$$9 \cdot \frac{n}{2} + 4$$

Hoca arada gözden kaçırdığı işlemler olabileceğini de söyledi. Sonuç olarak `while` sürümü daha az işlem yapar. Bellekte ise ikisi de ek dizi kullanmaz. `for` sürümünde `temp` ve `i`, `while` sürümünde `t`, `left` ve `right` vardır, yani yalnızca bir değişken fazladır.

İkisi de kabaca n mertebesindedir. Gerçek farkı görmek için hoca, 100 elemanlı ve 10 milyon elemanlı dizilerde aynı bilgisayarda süre ölçmeyi önerdi.

> **Ders sonrası not:** Hocanın sayımında `for` sürümünün döngü denetimi sayılmış, `while` sürümündeki `left < right` karşılaştırması ise büyük olasılıkla atlanmıştır. Bu eklenirse `while` sürümü $9 \cdot \frac{n}{2} + \left(\frac{n}{2} + 1\right) + 3 = 5n + 4$ civarı olur. `for` sürümünün toplamı $7n + 4$'tür. Sonuç değişmez, `while` sürümü yine daha az işlem yapar. Ayrıca eleman sayısı çiftse `left` ve `right` büyük olasılıkla hiç eşit olmaz, birbirini geçerek döngüden çıkar. `left < right` şartı bu durumu da karşılar.

8. Çoğunluk Elemanı (Majority Element)

8.1 Problem

Soru önce "dizide en sık geçen elemanın bulunması" diye okundu. Asistanlar ise çoğunluk oylaması (majority voting) yaklaşımı sunmuştu. Hoca soru metnine bakınca kastın şu olduğunu gördü: Çoğunluk elemanı, dizide $n/2$'den fazla geçen elemandır. Yani en sık geçmek yetmez, ayrıca yarıyı da geçmek gerekir.

Slayttaki birinci örnekte 8 elemanlı dizide 3, 5 kez geçer. $5 > \frac{8}{2}$ olduğu için çoğunluk elemanı 3'tür. İkinci örnekte 5 elemanlı dizide en sık geçen eleman 2 kez geçer. $2 > \frac{5}{2}$ sağlanmadığı için çoğunluk yoktur. Hoca bunun bir ek kontrol olduğunu söyledi.

8.2 Birinci Çözüm: İç İçe Döngü

Asistanların ilk çözümü, daha önce parti örneğinde yapılan iç içe döngüdür. Her eleman için bütün dizi gezilip frekansı sayılır. Frekans `maxCount`'tan büyükse `maxCount` ve indis güncellenir. Ek dizi yoktur. Hoca kodu ekrandan gösterdi, aşağıdaki hali anlatımdan tamamlanmıştır (`do-while` bir sonraki kodda açıklanıyor):

```c
#include <stdio.h>
#define MAX 100

int main(void)
{
    int dizi[MAX];
    int n, i, j, count, maxCount = 0, index = 0;

    do {
        printf("Enter size: ");
        scanf("%d", &n);
    } while (n > MAX);

    for (i = 0; i < n; i++) {
        printf("Eleman giriniz: ");
        scanf("%d", &dizi[i]);
    }
    for (i = 0; i < n; i++) {
        count = 0;
        for (j = 0; j < n; j++) {   /* kendisi de dahil */
            if (dizi[i] == dizi[j])
                count++;
        }
        if (count > maxCount) {
            maxCount = count;
            index = i;
        }
    }
    if (maxCount > n / 2)
        printf("Majority: %d\n", dizi[index]);
    else
        printf("No majority\n");
    return 0;
}
```

İki döngü de 0'dan n'e gittiği için `if` $n \cdot n$ kez çalışır. Bu n^2 karmaşıklıkta bir çözümdür, yanındaki basit işlemlerle $n^2 + n$ civarıdır.

8.3 İkinci Çözüm: Oylama Algoritması

İkinci yaklaşım hocanın "güzel bir algoritma" dediği, altyazıda "Morse Voting" diye geçen oylama algoritmasıdır. Önce çoğunluk adayı olan bir eleman belirlenir, sonra bu adayın frekansı hesaplanıp $n/2$'den büyük olup olmadığına bakılır. 100 elemanlı bir dizide adayın frekansı 50'yi geçiyorsa diğer elemanlara bakmaya gerek kalmadan o elemanın çoğunluk olduğu söylenebilir.

> **Ek bilgi (derste anlatılmadı):** Bu algoritmanın adı büyük olasılıkla Boyer-Moore çoğunluk oylamasıdır (Moore voting), altyazıdaki "Morse" bunun yanlış çözümlenmiş halidir. Çalışma mantığı büyük olasılıkla şöyle düşünülebilir: Sayacın her azalışı, farklı iki elemanın birbirini götürmesi demektir. Yarıdan fazla geçen bir elemanın bütün kopyaları diğer elemanlarla götürülemez, bu yüzden sonda aday olarak o kalır. Sayaç arada sıfırlanabilir, ama son adayı bu değiştirmez.

Hoca kodu derste yazdı. Dizinin boyutu kullanıcıdan alınır ve `MAX`'ı aşmaması gerekir. Hoca bunun için hangi döngünün uygun olduğunu sordu, cevap `do-while` idi. Önce n girmek denenir,

şart olarak da sohbette öğrencilerin yazdığı gibi `n > MAX` yazılır. Büyük girildiği sürece döngü tekrar eder.

Kodun tamamı şöyledir:

```
#include <stdio.h>
#define MAX 100

int main(void)
{
    int dizi[MAX];
    int n, i, maxi, count, candidate;

    do {
        printf("Enter size: ");
        scanf("%d", &n);
    } while (n > MAX);

    for (i = 0; i < n; i++) {
        printf("Eleman giriniz: ");
        scanf("%d", &dizi[i]);
    }

    /* 1. asama: adayi bul */
    maxi = 0;
    count = 1;
    for (i = 1; i < n; i++) {
        if (dizi[maxi] == dizi[i])
            count++;
        else
            count--;
        if (count == 0) {
            maxi = i;
            count = 1;
        }
    }

    /* 2. asama: adayin gercek frekansini say */
    candidate = dizi[maxi];
    count = 0;
    for (i = 0; i < n; i++) {
        if (dizi[i] == candidate)
            count++;
    }

    if (count > n / 2)
        printf("element %d, frequency %d\n", candidate, count);
    else
        printf("no majority\n");
    return 0;
}
```

Hoca `do-while`'ın sonundaki noktalı virgülün zorunlu olduğunu hatırlattı. Döngü satırının sonuna noktalı virgül konmamasının tek istisnası budur, normal `while` ile karışmaması için yazılır.

Soru: `while`'da da noktalı virgül kullanılıyor ama?

Cevap: Normal `while`'ın sonuna konursa döngünün gövdesi boş kalır, içerideki işler döngüde çalışmaz. Bir şey yaptırmak istiyorsanız kullanmazsınız. Hoca bunu boş `while` döngüsü olarak daha önce anlattığını söyledi.

Ders sonrası not: Derste koddaki son şart `count >= n / 2` yazıldı. Çoğunluk

> “$n/2$’den fazla” olduğu için doğrusu büyük olasılıkla `count > n / 2`’dir. 6 elemanlı bir dizide tam 3 kez geçen eleman çoğunluk değildir. Hoca bu noktayı derste “$n/2$’ye çok takılmaya gerek yok” diyerek geçti. Yukarıdaki kod `>` ile yazıldı ve `gcc -Wall -std=c89 -pedantic` ile uyarısız derlendi.

Derste `2 3 3 4 3 3` girildi. Ekranda 4 yazdı, yani 3’ün frekansı 4’tür. $4 > \frac{6}{2}$ olduğu için 3 çoğunluk elemanıdır. Program ders sonrasında da aynı sonucu verdi: `element 3, frequency 4`. Hoca daha sonra çıktıya elemanı da ekledi.

8.4 Adım Adım İzleme

Bazı şeyleri böyle analiz ederek daha iyi anlarız diyen hoca, $n = 8$ için `6 3 3 8 2 6 3 3` dizisini izledi. Başlangıçta `maxi = 0`, `count = 1`, aday 6’dır. Her satır o adım bittikten sonraki durumu gösterir:

`i`	`dizi[i]`	Adayla eşit mi?	`count` işlemi	`maxi`	Aday	`count`
1	3	Hayır	0’a düştü, aday değişti	1	3	1
2	3	Evet	arttı	1	3	2
3	8	Hayır	azaldı	1	3	1
4	2	Hayır	0’a düştü, aday değişti	4	2	1
5	6	Hayır	0’a düştü, aday değişti	5	6	1
6	3	Hayır	0’a düştü, aday değişti	6	3	1
7	3	Evet	arttı	6	3	2

Sonunda `maxi = 6`, `count = 2`’dir ve aday 3 bulunur. Sonra aday bütün dizide sayılır: 3, 4 kez geçer.

> **Ders sonrası not:** Bu dizide $4 > \frac{8}{2}$ sağlanmaz. Dersteki `>=` şartı yüzünden büyük olasılıkla program 3’ü sonuç olarak yazar, ama tanıma göre bu dizide çoğunluk elemanı yoktur. Düzeltilmiş kod `no majority` yazdı.

8.5 Algoritma En Sık Geçen Elemanı Bulur mu?

Hoca izleme sırasında “en fazla geçeni buluyor” dedi, ama aynı anda bunun denenmesi gerektiğini de ekledi.

Soru: Başta dört tane aynı eleman üst üste olsa, sonraki yedi tanesi hep farklı olsa aday yanlış çıkmaz mı?

Cevap: Hoca sohbetten gelen öneriyle bir dizi yazıp izledi. Baştaki aynı elemanlar sayacı yükseltir, sonraki farklı elemanlar sayacı düşürür. Sayaç 0 olunca aday değişir ve baştaki frekans unutulur.

Hocanın izlediği adımlar `3 3 3 4 5 6 7 8 9 10` dizisine uyar ($n = 10$):

`i`	`dizi[i]`	`count` işlemi	`maxi`	Aday	`count`
1	3	arttı	0	3	2
2	3	arttı	0	3	3
3	4	azaldı	0	3	2
4	5	azaldı	0	3	1

i	dizi[i]	count işlemi	maxi	Aday	count
5	6	0'a düştü, aday değişti	5	6	1
6	7	0'a düştü, aday değişti	6	7	1
7	8	0'a düştü, aday değişti	7	8	1
8	9	0'a düştü, aday değişti	8	9	1
9	10	0'a düştü, aday değişti	9	10	1

Son aday dizinin son elemanı olan 10'dur ve frekansı 1'dir. Oysa en sık geçen eleman 3'tür. Demek ki algoritma dizinin en sık geçen elemanını her zaman bulmaz.

Bir öğrenci meselenin tam olarak nerede bittiğini söyledi: Bu dizi için program zaten "no majority" diyecektir. 3'ün çoğunluk olması için yarıdan fazla geçmesi gerekir. Baştaki 3 sayısı yeterince fazlaysa sayaç sıfırlanmadan aday 3 kalır. Hoca bu tespiti doğruladı: $n/2$ şartına göre bakınca algoritma makuldür.

Hoca sonunda algoritmanın çıkış noktasının $n/2$ şartı olduğunu söyledi. Şart varken azalta azalta belli bir noktaya gelinir ve döngü tamamlanır. Yalnızca en sık geçen eleman istense bu yöntem yetmez.

> **Ders sonrası not:** Öğrenci 10 elemanlı dizi için "en az 5 tane" demiş, hoca da "sayaç sıfıra hiç ulaşmaz" diye onaylamıştı. Çoğunluk için büyük olasılıkla en az 6 tane gerekir. Baştaki beş 3'ten sonra beş farklı değer gelirse sayaç son elemanda 0'a düşer ve aday değişir. Python ile izlendiğinde son eleman aday oldu. Bu da yanlış sonuç değildir, çünkü 10 elemanda 5 kez geçen eleman çoğunluk sayılmaz. Ayrıca sayacın sıfırlanmaması yalnızca baştaki kopyalar bu kadar çoksa geçerlidir. Çoğunluk elemanı dizinin içine dağılmışsa sayaç arada sıfırlanabilir, son aday yine de doğru çıkar.

8.6 Tek Yedek Değişkenle En Sık Elemanı Bulma Denemesi

Aradan sonra hoca $n/2$ şartı olmadan en sık elemanı bulmak için aklına gelen bir yolu denedi. Dizi `4 8 3 4 6 3 4 4 2 5 0` olsun. İlk eleman majör kabul edilir, sayacı 1'dir. Bir de yedek majör ve yedek sayaç tutulur. Eleman majöre eşitse sayaç artar. Değilse mevcut majör yedeğe atılır ve yeni eleman majör olur:

Gelen eleman	Majör	Yedek
Başlangıç	4	yok
8	8	4
3	3	8
4	4	3

Dördüncü adımda 4 tekrar geldiğinde, ilk 4'ün frekansı artık hiçbir yerde yoktur, silinmiştir. Hoca bundan şu sonucu çıkardı: Tek değişkenle olmaz. Ya bir diziye ihtiyaç vardır ya da iç içe iki döngüyle kontrol etmek gerekir.

Hoca aklındaki fikrin, daha önce geliştirdiği başka bir algoritmaya benzediğini söyledi: Dizide arka arkaya en uzun süre değişmeden devam eden elemanın bulunması. Buna "longest common subsequence" dedi.

> **Ders sonrası not:** "Longest common subsequence" adı büyük olasılıkla iki dizinin en uzun ortak alt dizisi problemine aittir. Hocanın anlattığı, tek dizide arka arkaya tekrar eden en uzun bölümü bulmaktır. Bu büyük olasılıkla "en uzun tekrar (longest run)" diye anılır ve dizi sıralı olmasa da tek döngüyle çözülür. Bu yüzden derste görüldüğü gibi dağınık tekrarları saymaya yaramaz.

Hoca problemin kendisinin basit olduğunu, ama iyi algoritmasını yazmanın emek istediğini de ekledi.

9. Transpoz, Döndürme ve Aynalama

9.1 Transpoz

Matrisin transpozunu almak, `A[i][j]` yerine `A[j][i]` yazmak demektir. Kodda yeni bir matris tanımlanır. $M \times N$ boyutlu bir matrisin transpozu $N \times M$ boyutludur. Matris okunurken `T[j][i] = A[i][j]` ile kopyalanır. Kare matriste iş kolaydır. Kare olmayan matriste döngü değişkenlerinin sınırlarını ona göre ayarlamak gerekir.

9.2 Döndürme ve Aynalama

Hoca bir çalışma sorusu olarak şu matrisi verdi ve 90 derece döndüren kodu istedi:

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

Bir görüntünün 90 derece döndürülmesi gibi düşünülebilir. Pikseller yeni yerlerine gider. Sonuç olarak “3, 2, 1, 6, 5, 4, 9, 8, 7” okudu ve bunun transpoz olmadığını, transpozda aynı sütunun 1, 2, 3 olarak kalacağını söyledi. Ayrıca öbür tarafa 90 derece döndürmeyi ve aynalamayı (mirror) da örnek verdi: 7, 8, 9, 4, 5, 6, 1, 2, 3. Hoca buna “180 derece mirror” dedi.

> **Ders sonrası not:** Okunan sayılar büyük olasılıkla sütun sütun okunmuştur, çünkü hoca karşılaştırmada transpozun ilk sütununu kullandı. Öyleyse sonuç saat yönünün tersine 90 derece döndürmedir. Satır satır okunursa $3\,2\,1/6\,5\,4/9\,8\,7$ yalnızca her satırın ters çevrilmesidir, yani döndürme değil aynalamadır. Python ile hesaplanan sonuçlar şöyledir. Saat yönünde 90 derece: $7\,4\,1/8\,5\,2/9\,6\,3$. Saat yönünün tersine 90 derece: $3\,6\,9/2\,5\,8/1\,4\,7$. Hocanın $7\,8\,9/4\,5\,6/1\,2\,3$ matrisi ise satırların yukarıdan aşağıya ters çevrilmesidir, 180 derece döndürme değildir. 180 derece döndürme $9\,8\,7/6\,5\,4/3\,2\,1$ verir.

10. İkili Matrisi Simetrik Yapmak

10.1 Problem

0 ve 1’lerden oluşan bir kare matrisin köşegene göre simetrik olması için kaç değişiklik yapmak gerekir? Bir değişiklik, bir hücreyi 0’dan 1’e ya da 1’den 0’a çevirmektir. Hoca slayttaki örneklerde karşılıklı hücreleri gösterdi. Birinci örnekte tek bir değişiklik yeterlidir. İkinci örnekte köşegenin iki yanında uyuşmayan üç hücre vardır. Ya alttakiler üsttekiler gibi ya üsttekiler alttakiler gibi yapılır, ayrıca bir yerde daha değişiklik gerekir.

Soru: İki üçgeni ayırsak, üst üçgende dönüp farklı olanları saysak olmaz mı?

Cevap: Hoca önce daha basit bir çözüm istedi. Öğrencilerin hızlı gitmesine şöyle takıldı: Siz maşallah “lep” demeden leblebiyi, hatta kuruyemiş dükkânını aldınız. En basit düşünce için simetrik matrisin tanımına dönmek gerekir.

10.2 Birinci Çözüm: Transpozla Karşılaştırma

> **Hocanın vurgusu:** Simetrik matrisin tanımı, matrisin transpozuna eşit olmasıdır. Basit çözüm buradan çıkar.

Matrisin transpozu alınır, iki matris eleman eleman karşılaştırılır ve farklar sayılır. Bir uyuşmazlık iki matrisin iki hücresinde görüleceği için, yani hem yukarıdan hem aşağıdan sayılacağı için, sonuç 2’ye bölünür. Asistanların kodu ekranda gösterildi. Aşağıda transpoz ve sayma bölümleri ekrandaki gibidir. Değişken tanımları ve boyutun okunması görünmediği için tamamlanmış, matrisi ekrana yazdıran bölüm çıkarılmıştır:

```
#include <stdio.h>
#define MAX 20

int main(void)
{
```

```
    int A[MAX][MAX], T[MAX][MAX];
    int n, i, j, flip;

    printf("Enter size: ");
    scanf("%d", &n);
    for (i = 0; i < n; i++) {
        for (j = 0; j < n; j++) {
            printf("element[%d][%d]:", i, j);
            scanf("%d", &A[i][j]);
        }
    }

    /* Transpoz */
    for (i = 0; i < n; i++)
        for (j = 0; j < n; j++)
            T[i][j] = A[j][i];

    /* Farklari say */
    flip = 0;
    for (i = 0; i < n; i++)
        for (j = 0; j < n; j++)
            if (T[i][j] != A[i][j])
                flip++;

    printf("Number of flip is:%d\n", flip / 2);
    return 0;
}
```

İki döngü bütün değerleri gezdiği için n^2 işlem yapılır. Yanında n^2 ek bellek de gerekir. Hoca kodların fonksiyonsuz yazıldığını, bunun sebebinin diğer grubun fonksiyonları henüz görmemiş olması olduğunu söyledi.

Soru: `while` ile yapsak, eşit değilse hemen çıksak daha iyi olmaz mı?

Cevap: Hayır. Burada yalnızca simetrik mi diye bakmıyoruz, simetrik hale getirmek için en az kaç dönüşüm gerektiğini sayıyoruz. Bu yüzden bütün hücrelere bakmak gerekir. Yalnızca simetrik olup olmadığını denetlemek isteseydik, ilk bozan durumda çıkan `while` daha güzel olurdu.

10.3 İkinci Çözüm: Yalnızca Bir Üçgeni Gezmek

Transpoz almadan köşegene göre bakılabilir. Hücrenin köşegene göre karşılığıyla karşılaştırılması yeterlidir. Hoca bunu tahtada şöyle gösterdi: Alt üçgendeki her hücre, üst üçgendeki karşılığıyla karşılaştırılır. Satır i için `j`, 0'dan `i`'ye kadar gider:

```
#include <stdio.h>
#define MAX 20

int main(void)
{
    int A[MAX][MAX];
    int n, i, j, flip = 0;

    printf("Enter size: ");
    scanf("%d", &n);
    for (i = 0; i < n; i++) {
        for (j = 0; j < n; j++) {
            printf("element[%d][%d]:", i, j);
            scanf("%d", &A[i][j]);
        }
    }
```

```
    for (i = 0; i < n; i++)
        for (j = 0; j < i; j++)
            if (A[i][j] != A[j][i])
                flip++;

    printf("%d\n", flip);
    return 0;
}
```

$i = 0$ iken iç döngüye hiç girilmez. $i = 1$ olunca `j = 0` için `A[1][0]` ile `A[0][1]` karşılaştırılır. Her çift tek kez sayıldığı için 2'ye bölmeye gerek yoktur. İki kod da `gcc -Wall -std=c89 -pedantic` ile uyarısız derlendi.

İşlem sayısı için temel işlem karşılaştırma alınır. Hoca toplamı $\frac{n(n+1)}{2}$, yani yaklaşık $\frac{n^2}{2}$ olarak yazdı. Birinci çözümde n^2 işlem ve iki tane n^2'lik bellek vardı, burada yaklaşık $\frac{n^2}{2}$ işlem ve yalnızca bir tane n^2'lik bellek vardır. Köşegene göre bakmak hem işlem hem bellek açısından daha verimlidir.

> **Ders sonrası not:** `j < i` olduğu için `i` üst sınıra dahil değildir ve hoca bu sırada "i'ler dahil değil" demişti. Toplam büyük olasılıkla $\sum_{i=0}^{n-1} i = \frac{n(n-1)}{2}$ olarak yazılmalıydı. Python ile sayıldığında bu değer tuttu. Sonuç yine yaklaşık $\frac{n^2}{2}$'dir.

10.4 Öğrenci Soruları

Soru: Üst üçgen ile alt üçgenin elemanlarını toplayıp farkına baksak bulamaz mıyız?

Cevap: Hoca önce önerinin olabileceğini düşündü, sonra kendisi bir karşıt örnek buldu. Bir yanda hücreler 1 ve 0, karşı yanda da 0 ve 1 olsun. Üst üçgenin toplamı 1, alt üçgenin toplamı da 1 olur. Fark 0'dır, ama karşılıklı hücreler uyuşmadığı için matris simetrik değildir. Bu yüzden toplam üzerinden gitmek olmaz.

Soru: Sol üst köşedeki ve sağ alt köşedeki hücreyi atlasak olur mu?

Cevap: Hoca, sonuncu için nasıl yapılacağını o an bilemediğini söyledi. `i`'yi 1'den başlatmayı düşündü, sonra böyle yapılırsa sıfırıncı satırla kontrolün kaçacağını, bu yüzden `i`'nin sıfır olması gerektiğini söyledi.

> **Ders sonrası not:** Hoca `i`'yi 1'den başlatmayı büyük olasılıkla yanlışlıkla reddetti. Bu kodda `i = 0` için iç döngü zaten hiç çalışmaz. Sıfırıncı satırdaki `A[0][j]` hücreleri, `i = j` turunda `A[j][0]` ile karşılaştırılır. Dolayısıyla `i = 1`'den başlamak aynı sonucu verir. Köşegen hücreleri ise `j < i` şartı nedeniyle zaten hiç karşılaştırılmaz.

Soru: `i`'yi 0'dan başlatıp `j`'yi `i + 1`'den başlatsak, `j < n`'e kadar gitsek olur mu?

Cevap: Olur. Bu üst üçgen üzerinden bakmaktır. Üstteki hücreler alttakilerle aynı şekilde karşılaştırılır.

Dersin Toparlaması

Bu derste önce fonksiyon yazmanın ölçüsü konuşuldu: Tekrar eden alt programlar, kısa olsalar bile fonksiyon olmalıdır. Matrislerin fonksiyona `int M[][SUTUN]` biçiminde gönderileceği görüldü. Seçim sistemi örneklerinde her durumda önce doğru veri yapısının seçilmesi gerektiği vurgulandı: tek oyda dizi, çoklu oyda ve puanda matris, toplamlar için yine dizi. İki sıralama algoritması karşılaştırıldı. Sayarak sıralama n^2 mertebesinde kaldı. Doğrudan yerleştirme çok hızlı, ama en büyük değer kadar bellek gerektirdi.

Uygulama sorularında ise aynı problemin farklı çözümleri işlem sayısı ve bellek bakımından tartıldı. Dizi ters çevirmede `while` sürümü, çoğunluk elemanında oylama algoritması, simetri probleminde tek üçgeni gezmek daha iyi çözümler oldu. Temel işlemi seçip toplam sembolleriyle saymak hepsinin ortak aracıydı. Bir sonraki derste yedinci uygulama sorusu ele alınacak, döngülerle yazılmış (iteratif) algoritmaların nasıl analiz edileceğine basit örneklerle bakılacak ve rastgele sayı üretimi konuşulacaktır.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.